Summary of "Hardware/Infrastructure"

Reviewers: Emily Chen (emychen), Marie Yu (yumarie)

Problem and Motivation

Memory bound LLM inference

Within LLM inference there are two phases: prefill and decode. Prefill uses GEMM (General Matrix to Matrix Communication) and decode uses GEMV (General Matrix Vector Multiplication). These operations mean that LLM inference is a memory-bandwidth bound operation. The presenters address a key insight: that most of today's systems rely on off-chip solutions, which are constrained by communication latency and high memory bandwidth. The GPU also constantly fetches models from off-chip memory, which wastes idle time. This is the problem that WaferLLM aims to solve.

Rearchitecting datacenters

LLM inference workloads are driving massive demand within cloud datacenters, however, traditional datacenter management was designed for CPU-centric/general-purpose workloads, which is fundamentally mismatched with the resource needs of inference workloads. The paper introduces a new metric: Total Cost of Ownership (TCO) and uses it to recommend best practices for datacenters that primarily serve AI inference.

Related Works

The presenters mention the shift in research from off-chip network memory to on-chip memory, bringing us to the more unexplored Wafer-scale AI architecture. Systems like Megatron-LM which use off-chip network memory were explored, and FlashAttention/PageAttention with on-chip shared memory have also been explored, but distributed, mesh-based memory has been largely unexplored.

Solution Overview

WaferLLM

WaferLLM aims to address the issues mentioned with high memory bandwidth and network communication latency within traditional off-chip LLM inference solutions. Instead of an off-chip network memory, WaferLLM uses a mesh network-on-chip (NoC) where cores communicate primarily with their immediate neighbors. The paper focuses on using Cerebras WS-2 and tries to adhere to the PLMR model (million scale **Parallelism**, highly non-uniform access **Latency**, constrained local **Memory**, and constrained routing **Resources**).

WaferLLM first uses new prefill parallelism strategies, with fine grained partitioning for million-core parallelism, PLMR-compliant distributed GEMM, and transposed distributed GEMM.

For the decode stage, the authors designed a fine-grained replication to enable parallelism at minimal communication cost, designed PLMR-compliant GEMV, and pre-optimized model weight placement.

For KV Cache management, they take advantage of the wafer based architecture. Since existing concatenate-based management causes skewed core utilization, they used shift-based management instead, allowing for balanced core utilization.

For matrix computation, instead of pipeline allreduce or ring allreduce, WaferLLM uses K-Tree allreduce, where cores are partitioned into K groups, and each group reduces in parallel. This creates a tradeoff of higher routing complexity for lower latency.

The authors evaluated WaferLLM against other hardware/memory solutions, such as T10, Ladder, and SGLang, finding significant speedups and energy savings on WaferLLM. They also measured throughput per requests for MeshGEMM/MeshGEMV algorithms against SUMMA and Cannon, finding that other solutions required more cycles to reach the same computations as the Mesh algorithms.

Rearchitecting Datacenters

The authors start by outlining the three stages of the lifecycle of datacenter management, which consist of:

1. Build: Initial infrastructure setup. This includes making decisions about power topology, cooling, networking, facility design.
2. IT provisioning: Governs when and how to decommission and upgrade hardware.
3. Operations: runtime management, workload placement, scheduling, and software optimizations.

The presenters then outline a TCO-driven lifecycle framework, with assumptions made that the datacenter serves primarily LLM inference and also assumes the linear growth of AI model sizes and memory bandwidth, power dissipation and cost. Specifically, a lifecycle evaluation consists of simulating the baseline policy, and seeing how the AI fleet changes over time as model sizes and demand increase. The model is run with many different possible futures depending on the scale and speed of growth in demand, GPU releases, and costs to compare. Taking long term sustainability and flexibility in mind, comparing these models allows them to find which models will most likely perform well in the future when exposed to uncertain events on demand and research advances.

The authors found that the architectural needs of datacenters serving traditional workloads vs AI inference workloads were distinct. First outlining power distribution, traditional workloads use hierarchical power distribution. However, due to high-end GPU servers drawing massive amounts of power compared to CPUs, the authors propose using flat per datacenter power delivery architecture.

They also found that for cooling solutions, 75% liquid cold plates and 25% air was the sweet spot, reducing TCO by 9% over a full lifecycle. For networking strategy, uniform strategies were more expensive and inefficient for AI workloads, finding that hierarchical strategies (NVLink within servers, InfiniBand within racks, and Ethernet across racks) were more optimal.

For provisioning AI hardware, the authors found that smaller models could run better on older GPUs, meaning that model size has a meaningful impact on decisions on when to upgrade GPUs. Sparsity also extends hardware life, as sparse model architectures degrade less on older hardware compared to dense counterparts. SSMs also offer hardware compatibility, and are more hardware efficient than transformers.

For refresh policies, there is also a distinction between tradition and AI workloads, as the 5 year fixed refresh cycle is suboptimal for AI. Dynamic/flexible strategies work better, with more recent refreshes being justified by massive efficiency gains. However, generation skipping is a viable strategy if GPUs offer marginal gains. These flexible policies can reduce TCO by 15-20% compared to the baseline.

Overall, the key insight is that while traditional datacenters typically scale linearly with new hardware, AI workloads result in larger variability. Different hardware gains vary dramatically between different models and use cases, so we can see great gains in implementing heterogeneity aware scheduling, infrastructure-aware scheduling, and phase disaggregation. Applying all of these improvements overall achieves over 40% total TCO reduction.

Limitations

While limitations were not explicitly mentioned in the presentation itself, the class discussion exposed some potential limitations in the current architecture and the experiments that the authors performed. For example, discussions on the KVCache upshift strategy revealed that the authors chose small models such as Llama 8B and LLama 13B, finding that for larger models they simply ran out of memory as they shifted older data from the KVCache up.

There was also discussion on energy and carbon as an objective, with the methodology for measuring energy usage in WaferLLM being unclear.

Future Research Directions

Further research directions were also more discussed within the class discussion, going hand in hand with the limitations of the studies. For example, a cache eviction/memory management strategy could be explored in the case of shift-based work. Further discussions on energy measurement and energy efficiency could also be explored, as this was discussed. More clarity/methods on how to explicitly measure energy usage for Cerebus W2 or similar mesh-based hardware could be explored, as well as the implications on efficiency for mesh-based architecture.

Summary of Class Discussion

Some of the class discussion has also been mentioned in the previous sections, such as discussions about the shift-based memory management for the KVCache. Specifically, it was asked what would happen to the oldest data at the very top of the hardware, and the answers gave insight into the limitations of the memory management system, where only small models were run to ensure the KVCache stayed within expected parameters. Other discussions were on whether long distance communications could be avoided or not, with the usage of WaferLLM making it likely that these long distance communications can be entirely avoided. Another consideration was whether increasing the number of cores could bottleneck communication again.

Some questions also revolved around energy efficiency, power and eco friendliness, such as how WaferLLM evaluates energy efficiency with the Cerebus W2 chip, and whether systems could be optimized for other variables, such as operational carbon. Some answers were that carbon measurements can be easy to obfuscate, and it is important to note what this measurement means and whether it is useful.

Lastly, it was noted that usage of the wafer chip makes programming with multithreaded programs different, and whether this changed the programming abstraction for multithreading and parallelism. While it is not explicitly stated in the paper what the abstraction would be, it is an interesting case to think about with these new mesh-based hardware and architectures.